AAC - BIOcean5D Coral Microbiomes: 05 - Community Analyses

Eric J. Armstrong, eja98@cornell.edu, @EricJ_Armstrong

28 October, 2026

Contents

Purpose of this file This file contains all of the code for producing a curated OTU table from the 16S microbial sequence data for the Armstrong et al. manuscript entitled **"XXX"**, published by XXX.

If you have any questions, you are welcome to email the corresponding author at eja98@cornell.edu or tweet him at EricJ_Armstrong. The goal is to perform community level diversity analyses

1. Load in phyloseq data with rooted tree.
2. Evaluate sequencing depth and remove sample.
3. Normalize the read counts between samples.
4. Calculate community dissimilarities. Numbers between 0 and 1. If 0, completely similar versus if they are 1, then they're completely dissimilar.
    a. **Sorensen**: Shared Species as a binary value: Abundance-unweighted
    b. **Bray-Curtis**: Shared Abundant species: Abundance-weighted
    c. **(Abundance-)Weighted UNIFRAC**: Consider Abundant Species and where they fall on the tree
5. Visualize the community data with two unconstrained Ordinations:
    a. **PCoA**: Linear Method. Eigenvalue = how much variation is explained by each axis. Choose to view axis 1, 2, 3, etc. and plot them together.
    b. **NMDS**: Non-linear. Smush multiple Dimensions into 2 or 3 axes. Need to report Stress value (ideally <0.15).
6. Run statistics with PERMANOVA and betadispR.

1 - Prepare Workspace

Load necessary R packages and set default parameters

1.1 - Define Paths

```
# a - Define the main project directory ----
  dir_proj <-
  "C:/Users/uax75/OneDrive/Documents/R/AAC-BIOcean5D-Coral_Microbiome/"

# b - Define the directory containing the input data ----
  dir_in <-
  "C:/Users/uax75/OneDrive/Documents/R/AAC-BIOcean5D-Coral_Microbiome/Data/"

# c - Define the directories that will contain output Files/Figures/Tables ----
  dir_out_dat <-
    "C:/Users/uax75/OneDrive/Documents/R/AAC-BIOcean5D-Coral_Microbiome/Data/05-AAC-CommunityAnalyses/"
  dir.create(dir_out_dat, recursive = TRUE)
  dir_out_fig <-
    "C:/Users/uax75/OneDrive/Documents/R/AAC-BIOcean5D-Coral_Microbiome/Figures/05-AAC-CommunityAnalyse
  dir.create(dir_out_fig, recursive = TRUE)
  dir_out_tab <-
    "C:/Users/uax75/OneDrive/Documents/R/AAC-BIOcean5D-Coral_Microbiome/Tables/05-AAC-CommunityAnalyses
  dir.create(dir_out_tab, recursive = TRUE)
```

1.2 - Set Defaults

```
# a - Define the output prefix for generated files ----
  pref_out <- "05-AAC-CommunityAnalyses_"

# b - Set the random seed value ----
  set.seed(123)
```

1.3 - Load Libraries

```
# a - load necessary packages ----
  pacman::p_load(tidyverse, devtools, phyloseq, patchwork, vegan,
                 ggplot2, pairwiseAdonis, nlme, car,
                install = FALSE)
```

```
# PATCH 2026-10-28: reusable between-colony test for Origin.
# Plots(strata = ..., type = "free") requires BALANCED strata - permute::check() stops with
# "Design must be balanced if permuting 'strata'" as soon as colonies contribute unequal
# fragment counts, which became true after the amplicon-window repair changed which
# libraries survive filtering. This helper keeps the whole-colony permutation wherever the
# design is still balanced, and otherwise collapses each colony to a centroid and tests
# Origin on those points under unrestricted permutation - a valid between-colony test that
# needs no balance, because one point per colony makes colony the unit of replication by
# construction. NOTE: in the centroid case R2 is the variance among COLONY CENTROIDS
# explained by Origin, not the variance among fragments, so it is not comparable with the
# balanced-case R2. The scheme actually used is printed and stored as an attribute.

origin_between_test <- function(dm, meta, nperm = 999) {
```

```
  stopifnot(all(c("Colony_ID", "Origin") %in% names(meta)))

  frag     <- table(meta$Colony_ID)
  balanced <- length(unique(as.vector(frag))) == 1

  if (balanced) {
    ctrl <- how(plots  = Plots(strata = meta$Colony_ID, type = "free"),
                within = Within(type = "none"), nperm = nperm)
    res  <- adonis2(dm ~ Origin, data = meta, permutations = ctrl, by = "terms")
    attr(res, "scheme") <- "whole colonies"
  } else {
    k    <- min(nrow(meta) - 1L, 20L)
    pc   <- cmdscale(dm, k = k, eig = TRUE)
    keep <- pc$eig[seq_len(ncol(pc$points))] > 1e-8
    pts  <- pc$points[, keep, drop = FALSE]
    cen  <- aggregate(pts, by = list(Colony_ID = meta$Colony_ID), FUN = mean)
    key  <- dplyr::distinct(meta[, c("Colony_ID", "Origin")])
    cen  <- dplyr::left_join(cen, key, by = "Colony_ID")
    dmc  <- dist(cen[, setdiff(names(cen), c("Colony_ID", "Origin"))])
    res  <- adonis2(dmc ~ Origin, data = cen, permutations = nperm, by = "terms")
    attr(res, "scheme") <- "colony centroids"
  }

  cat("  fragments per colony:",
      paste(sort(unique(as.vector(frag))), collapse = "/"),
      "->", attr(res, "scheme"), "\n")
  res
}
```

1.4 - Load Phyloseq Object

```
# a - Load physeq ----
  load(paste(dir_in,
             "03-AAC-PhylogeneticTree/",
             "03-AAC-PhylogeneticTree_physeq_midpointTree.RData",
             sep = ""))

# b - Perform targeted curation: Establish clean symmetric 3-node timeline ----
  metadata_raw <- data.frame(sample_data(physeq_midpointTree)) %>%
    mutate(SampleID_Temp = rownames(.)) %>%
    # Force categorical fields to text strings to prevent factor integer-masking bugs
    mutate(
      Treatment  = as.character(Treatment),
      Transplant = as.character(Transplant),
      Host       = as.character(Host),
      Type       = as.character(Type)
    ) %>%
    # 1. Drop 2-Week samples completely to establish inter-species symmetry
    filter(Treatment != "2-Weeks" | is.na(Treatment)) %>%
    # 2. NA-PROOF RECIPIENT FILTER: Only clean rows if they are true Corals
    # This prevents R from deleting environmental references (water/sediment) due to NA cells
```

```
    filter(Type != "Coral" | !(Host == "POC" & Treatment == "Genotype" & Transplant != "Genotype2")) %>%
    # PATCH 2026-10-26: preserve the ORIGINAL sampling timepoint before recoding.
    # The case_when below folds Pre-HS into "4-Month-Pre-HS", which silently empties any
    # later subset asking for Treatment %in% c("Pre-HS", "Post-HS").
    mutate(Timepoint = as.character(Treatment)) %>%
    # 3. Safe mapping across symmetric nodes for all remaining samples
    mutate(Treatment = case_when(
      Type == "Coral" & Transplant == "Genotype2"               ~ "Genotype",
      Type == "Coral" & Treatment %in% c("4-Months", "Pre-HS")  ~ "4-Month-Pre-HS",
      Type == "Coral" & Treatment == "Post-HS"                  ~ "Post-HS",
      TRUE                                                      ~ Treatment
    )) %>%
    # Convert back into factors for downstream statistical modeling
    mutate(
      Treatment  = as.factor(Treatment),
      Transplant = as.factor(Transplant),
      Host       = as.factor(Host),
      Type       = as.factor(Type)
    )

  # Re-apply clean metadata subset matrix back onto parent phyloseq infrastructure
  physeq_pruned <- prune_samples(metadata_raw$SampleID_Temp, physeq_midpointTree)
  sample_data(physeq_pruned) <- sample_data(metadata_raw)

# c - Add a column to the metadata listing the full treatment condition ----
  sample_data(physeq_pruned)$FullTreatment <- paste(
    sample_data(physeq_pruned)$Site,
    sample_data(physeq_pruned)$Subtype,
    sample_data(physeq_pruned)$Transplant,
    sample_data(physeq_pruned)$Treatment,
    sep = " "
  )

# d - Fix the duplicate colony labels by combining Host and Colony names ----
  metadata_clean <- as(sample_data(physeq_pruned), "data.frame") %>%
    mutate(
      # DEPRECATED 2026-10-26: retained only for reference - it omits the bay. Use Colony_ID.
      Unique_Colony = case_when(
        Type == "Coral" ~ paste0(Host, "_", Colony),
        TRUE ~ "Environmental_Ref"
      ),
      # PATCH 2026-10-26: Unique_Colony above omits the bay, so colony CO1 from Tiahura and
      # colony CO1 from Opunohu collapse into one block even though they are different colonies.
      # Colony_ID is the corrected identifier; Site here is the colony's bay of ORIGIN.
      Colony_ID = case_when(
        Type == "Coral" ~ paste0(Host, "_", Site, "_", Colony),
        TRUE ~ "Environmental_Ref"
      ),
      # PATCH 2026-10-26: Site is the ORIGIN bay. Derive where the fragment actually was.
      Origin = as.character(Site),
      Destination = case_when(
        Type != "Coral"                ~ as.character(Site),
        Transplant == "Out Transplant" ~ if_else(Site == "Tiahura", "Opunohu", "Tiahura"),
```

```
        TRUE                            ~ as.character(Site)
      )
    )

# e - Re-assign the corrected metadata back into your workspace tree object ----
  sample_data(physeq_pruned) <- sample_data(metadata_clean)
  physeq_midpointTree <- physeq_pruned

  sample_data(physeq_midpointTree)$Colony <- paste(
    sample_data(physeq_midpointTree)$Site,
    sample_data(physeq_midpointTree)$Host,
    sample_data(physeq_midpointTree)$Colony,
    sep = ""
  )

  sample_data(physeq_midpointTree)$Transplant <-
    gsub("Genotype2",
         "Genotype",
         sample_data(physeq_midpointTree)$Transplant)

# # a - Load physeq ----
#   load(paste(dir_in,
#              "03-AAC-PhylogeneticTree/",
#              "03-AAC-PhylogeneticTree_physeq_midpointTree.RData",
#              sep = ""))
#
# # b - Add a column to the metadata listing the full treatment condition ----
#   sample_data(physeq_midpointTree)$FullTreatment <- paste(
#     sample_data(physeq_midpointTree)$Site,
#     sample_data(physeq_midpointTree)$Subtype,
#     sample_data(physeq_midpointTree)$Transplant,
#     sample_data(physeq_midpointTree)$Treatment,
#     sep = " "
#   )
#
# # c - Fix the duplicate colony labels by combining Host and Colony names ----
#   # Also handle NA values for environmental samples safely
#   metadata_clean <- as(sample_data(
#     physeq_midpointTree), "data.frame") %>%
#     mutate(
#       Unique_Colony = case_when(
#         Type == "Coral" ~ paste0(Host, "_", Colony),
#         TRUE ~ "Environmental_Ref" # Labels water and sediment uniformly
#       )
#     )
#
# # d - Re-assign the corrected metadata back into your scaled phyloseq object ----
#   sample_data(physeq_midpointTree) <- sample_data(metadata_clean)
```

2 - Explore Read Counts and Remove Low Count Samples

2.1 - Raw read depth

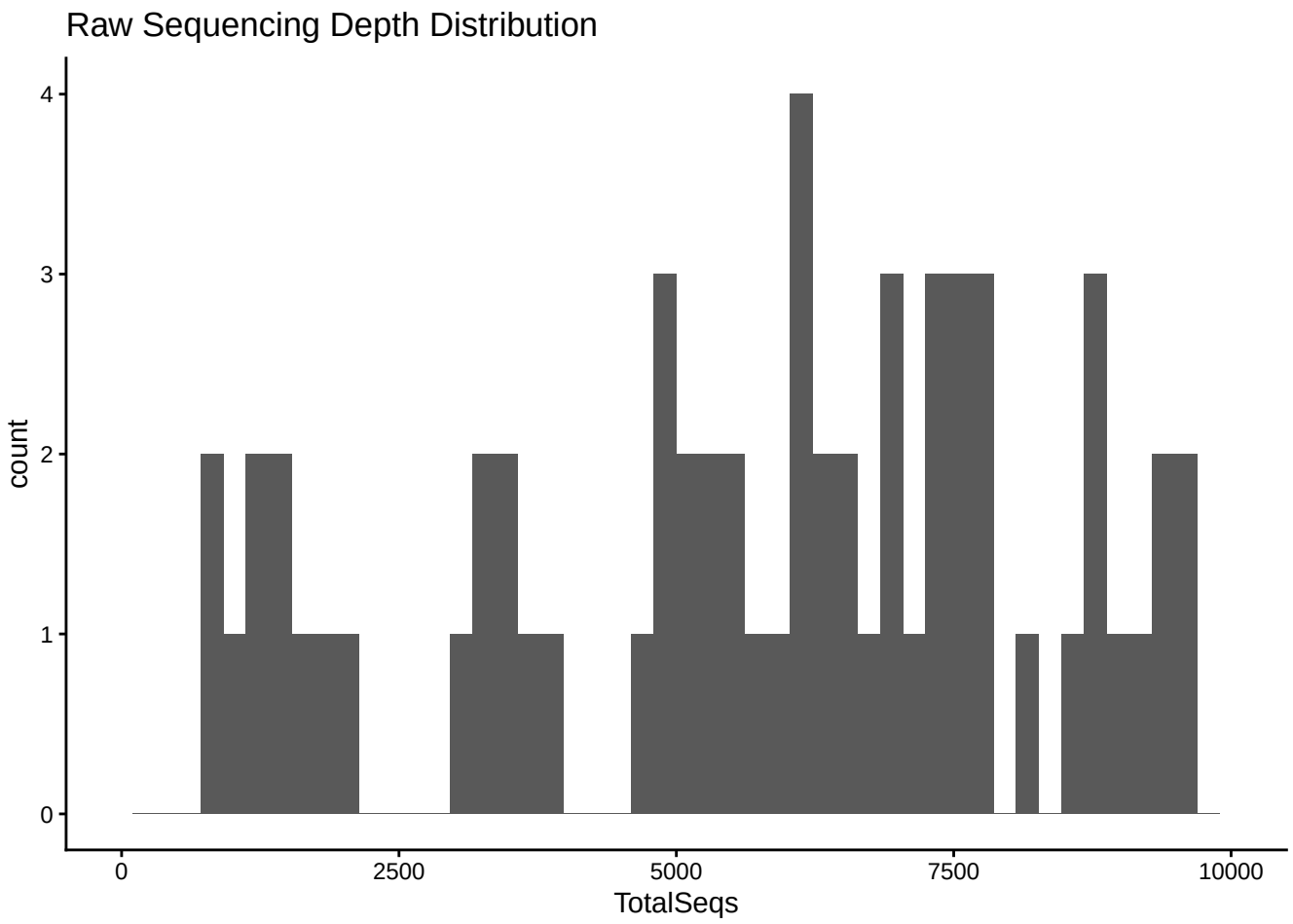
```
# a - Calculate the total number of reads per sample ----
  df_raw_TotalSeqs  <-
    physeq_midpointTree %>%
    # calculate the sample read sums
    sample_sums() %>%
    data.frame()

# b - name the column ----
  colnames(df_raw_TotalSeqs)[1] <- "TotalSeqs"

  head(df_raw_TotalSeqs)
#>                 TotalSeqs
#> 100-TBC01F60POC      5471
#> 101-TBC02F60POC      7810
#> 102-TBC03F60POC      9069
#> 103-TBC04F60POC      8870
#> 104-TBC05F60POC     11177
#> 105-TTC01F80POR     20935
```

2.2 - Visualize the raw read depth

```
# a - make histogram of raw reads ----
  df_raw_TotalSeqs %>%
    ggplot(aes(x = TotalSeqs)) +
    geom_histogram(bins = 50) +
    scale_x_continuous(limits = c(0, 10000)) +
    labs(title = "Raw Sequencing Depth Distribution") +
    theme_classic()
```



2.3 - Determine Minimum Read Depth and Remove Low Sequence Samples

```
# a - what is the minimum number of sequences? ----
  physeq_midpointTree %>%
    sample_sums() %>%
    min()
#> [1] 754

# b - Visualize the rarefaction curves to assess coverage ----

  # Extract the OTU table (vegan requires species as columns, samples as rows)
  # FORCE phyloseq to export with samples as rows
    if (taxa_are_rows(physeq_midpointTree)) {
      OTU_table <- as.data.frame(t(otu_table(physeq_midpointTree)))
    } else {
      OTU_table <- as.data.frame(otu_table(physeq_midpointTree))
    }

  # Generate the rarefaction curves
  # Note: 'step' dictates sampling intervals; 'label = FALSE' suppresses row name tags if they clutter
  plot_rareCurves <-
```